

MERN

Step 1:

```
mkdir mern-student-app
```

```
cd mern-student-app
```

Step 2: Folder structure

```
mern-student-app
```

```
|  
├── backend  
└── frontend
```

```
cd backend
```

```
npm init -y
```

step 3:

Install

```
npm install express mongoose cors nodemon
```

4: Make following structure

```
backend
```

```
|  
├── models  
|   └── Student.js  
|  
├── routes  
|   └── studentRoutes.js  
|  
├── server.js  
└── package.json
```

Student.js

```
const mongoose = require("mongoose");

const StudentSchema = new mongoose.Schema({
  name: String,
  email: String
});

module.exports = mongoose.model("Student", StudentSchema);
```

studentRoutes.js

```
const express = require("express");
const router = express.Router();
const Student = require("../models/Student");
```

// CREATE

```
router.post("/", async (req, res) => {
  const student = new Student(req.body);
  const saved = await student.save();
  res.json(saved);
});
```

// READ

```
router.get("/", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});
```

// UPDATE

```
router.put("/:id", async (req, res) => {
```

```

try {
  const updated = await Student.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );

  res.json(updated);
} catch (err) {
  res.status(500).json(err.message);
}
});

// DELETE
router.delete("/:id", async (req, res) => {
  try {
    await Student.findByIdAndDelete(req.params.id);
    res.json({ message: "Student deleted" });
  } catch (err) {
    res.status(500).json(err.message);
  }
});

module.exports = router;

```

```

package.json
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",

```

```
"start": "nodemon server.js",
"dev": "nodemon server.js"
},
"keywords": [],
"author": "",
"license": "ISC",
"type": "commonjs",
"dependencies": {
  "axios": "^1.14.0",
  "cors": "^2.8.6",
  "express": "^5.2.1",
  "mongoose": "^9.4.1"
},
"devDependencies": {
  "nodemon": "^3.1.14"
}
}
```

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

console.log("Step 1: imports loaded");

const studentRoutes = require("./routes/studentRoutes");

console.log("Step 2: routes loaded");

const app = express();

app.use(cors());
```

```
app.use(express.json());

console.log("Step 3: middleware added");

// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/studentDB")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log("Mongo Error:", err));

app.use("/api/students", studentRoutes);

console.log("Step 4: before listen");

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Download MongoDB Community Server:

<https://www.mongodb.com/try/download/community>

During install:

- ✓ Choose **Complete**
- ✓ Install MongoDB as Service (IMPORTANT)

```
npm install mongoose
```

MongoDB creates DB automatically when data is inserted.

Install MongoDB Compass (GUI):

<https://www.mongodb.com/products/compass>

Open Compass → connect using:

mongodb://127.0.0.1:27017

You will later see:

studentDB

STEP 1 — Find MongoDB Installation Folder

Open File Explorer and go to:

C:\Program Files\MongoDB\Server\

You should see a version folder like:

8.0

7.0

6.0

Open it → then open:

bin

Final path will look like:

C:\Program Files\MongoDB\Server\8.0\bin

 Copy this full path.

Add MongoDB to PATH

Open Environment Variables

Press:

Windows Key + S

Search:

Environment Variables

Click:

Edit the system environment variables

Click

Environment Variables

Under System Variables

Find:

Path

Select → Click Edit

Click New

Paste:

C:\Program Files\MongoDB\Server\8.0\bin

(Use your version number.)

Click OK → OK → OK

npm run dev

frontend

└─ src

│ └─ App.jsx

└─ components

│ └─ StudentForm.jsx

Step 1 — Create React App

Open NEW terminal (outside backend):

```
cd mern-project  
npx create-vite
```

Choose:

```
React  
JavaScript
```

Then:

```
cd frontend  
npm install  
npm run dev
```

Open browser:

<http://localhost:5173>

React running.

Step 2 — Install Axios (API calls)

Inside frontend:

```
npm install axios
```

Step 3 — Clean React Structure

Inside frontend/src delete:

```
App.css  
assets/
```

Step 4 — Create Components

Create folder:

```
src/components
```

Create file:

StudentForm.jsx

```
import { useState, useEffect } from "react";
```



```
import axios from "axios";
```

```
function StudentForm() {
```

```
  const [name, setName] = useState("");
```

```
  const [email, setEmail] = useState("");
```

```
  const [students, setStudents] = useState([]);
```

```
  const [editId, setEditId] = useState(null);
```

```
  // FETCH
```

```
  const fetchStudents = async () => {
```

```
    const res = await axios.get("http://localhost:5000/api/students");
```

```
    setStudents(res.data);
```

```
  };
```

```
  useEffect(() => {
```

```
    fetchStudents();
```

```
  }, []);
```

```
  // CREATE or UPDATE
```

```
  const handleSubmit = async (e) => {
```

```
    e.preventDefault();
```

```
    if (editId) {
```

```
      // UPDATE
```

```
      await axios.put(
```

```
        `http://localhost:5000/api/students/${editId}`,
```

```
        { name, email }
```

```
      );
```

```
      setEditId(null);
```

```

    } else {
      // CREATE
      await axios.post("http://localhost:5000/api/students", {
        name,
        email,
      });
    }

    setName("");
    setEmail("");
    fetchStudents();
  };

  // DELETE
  const handleDelete = async (id) => {
    await axios.delete(`http://localhost:5000/api/students/${id}`);
    fetchStudents();
  };

  // EDIT
  const handleEdit = (student) => {
    setName(student.name);
    setEmail(student.email);
    setEditId(student._id);
  };

  return (
    <div style={{ padding: "20px" }}>
      <h2>{editId ? "Update Student" : "Add Student"}</h2>

```

```
<form onSubmit={handleSubmit}>
  <input
    placeholder="Name"
    value={name}
    onChange={(e)=>setName(e.target.value)}
  />
```

```
  <input
    placeholder="Email"
    value={email}
    onChange={(e)=>setEmail(e.target.value)}
  />
```

```
  <button type="submit">
    {editId ? "Update" : "Add"}
  </button>
</form>
```

```
<h2>Student List</h2>
```

```
<ul>
  {students.map((s) => (
    <li key={s._id}>
      {s.name} — {s.email}

      <button onClick={()=>handleEdit(s)}>
        Edit
      </button>
    )
  )}
```

```

        <button onClick={()=>handleDelete(s._id)}>
            Delete
        </button>
    </li>
    )}
</ul>
</div>
);
}

```

```
export default StudentForm;
```

App.jsx

```

import StudentForm from "../components/StudentForm";

function App() {
    return (
        <div>
            <h1>MERN Student App</h1>
            <StudentForm />
        </div>
    );
}

```

```
export default App;
```

Test using postman

Open Postman (or Thunder Client).

Request:

POST <http://localhost:5000/api/students>

Body → JSON:

```
{  
  "name": "Test User",  
  "email": "test@gmail.com"  
}
```

Click Send.

Execute on two different terminals for frontend and backend.

Frontend /react

<http://localhost:5173>

For backend

<http://localhost:5000/api/students>